

Presentation Notes: Milestone 2

Reverse Proxy & Security Hardening

Devile Théo, Reynders Erwan, Bricusse Tristan

April 2026

1 Introduction

For Milestone 2, our main goal was to secure the network perimeter. We deployed a Reverse Proxy between routers R1 and R3. The core idea was to make internal web services accessible from the Internet without ever exposing the internal servers directly.

2 Architecture: The Reverse Proxy as a Middlebox

We used a Debian-based NGINX Reverse Proxy. Here is the theoretical justification for this setup:

- **Network Segregation:** The proxy acts as a Middlebox. It intercepts all external traffic, breaking the direct IP connection.
- **Zero Trust Principle:** By placing this proxy in the DMZ, we apply the "Least Privilege" rule. The untrusted external client (Malicious PC) can only talk to the proxy, keeping the "Trusted Zone" completely hidden.

3 Security Hardening (WAF Implementation)

We transformed the basic proxy into a Web Application Firewall (WAF) by implementing several security features.

3.1 1. Encryption and Integrity (SSL/TLS)

To secure the data in transit between the client and our proxy:

- We implemented SSL/TLS to ensure **Confidentiality** (via encryption) and **Integrity** (via hashing).
- The identity of the server is verified through secure key exchanges and certificates, preventing unauthorized interception.

3.2 2. Traffic Control (DoS Mitigation)

To ensure our services stay available and aren't overwhelmed:

- **Rate Limiting:** We restricted the number of requests per IP. This is our primary defense against Denial of Service (DoS) and resource exhaustion attacks.
- **Size Limits:** Limiting the request body size protects the servers from buffer overflow attempts.

3.3 3. Information Hiding (Anti-Reconnaissance)

A crucial part of our defense is preventing the attacker from gathering information:

- **Hiding Server Version:** By masking the NGINX version, we block the "Footprinting" phase. If an attacker doesn't know our software version, they cannot easily exploit known CVEs.
- **Secure Headers:** We configured headers like HSTS to force HTTPS, preventing Man-in-the-Middle (MitM) attacks, and other headers to block client-side attacks like Cross-Site Scripting (XSS).

4 Conclusion

In summary, our implementation for Milestone 2 successfully applies the theoretical concepts of network segregation, cryptography, and access control to create a resilient perimeter against standard external threats.